

Arbeiten mit entfernten Repositories auf GitHub

Ein praktischer Leitfaden zu lokalen und Remote-Repositories, Synchronisation, Zusammenarbeit und Dokumentation mit Git und GitHub.



Lokales vs. Remote-Repository

Das wichtigste Grundbild beim Arbeiten mit Git und GitHub: Es gibt zwei Orte, an denen dein Projekt lebt.

Lokal = Arbeitsplatz

Dein Repository liegt auf deinem Rechner. Dort bearbeitest du Dateien, machst Commits und experimentierst in Ruhe.

Remote = Zentraler Ablageort

Das Repository liegt auf einem Server, z. B. GitHub. Dort wird dein Projekt online gespeichert, geteilt und synchronisiert.

Git = Versionskontrolle

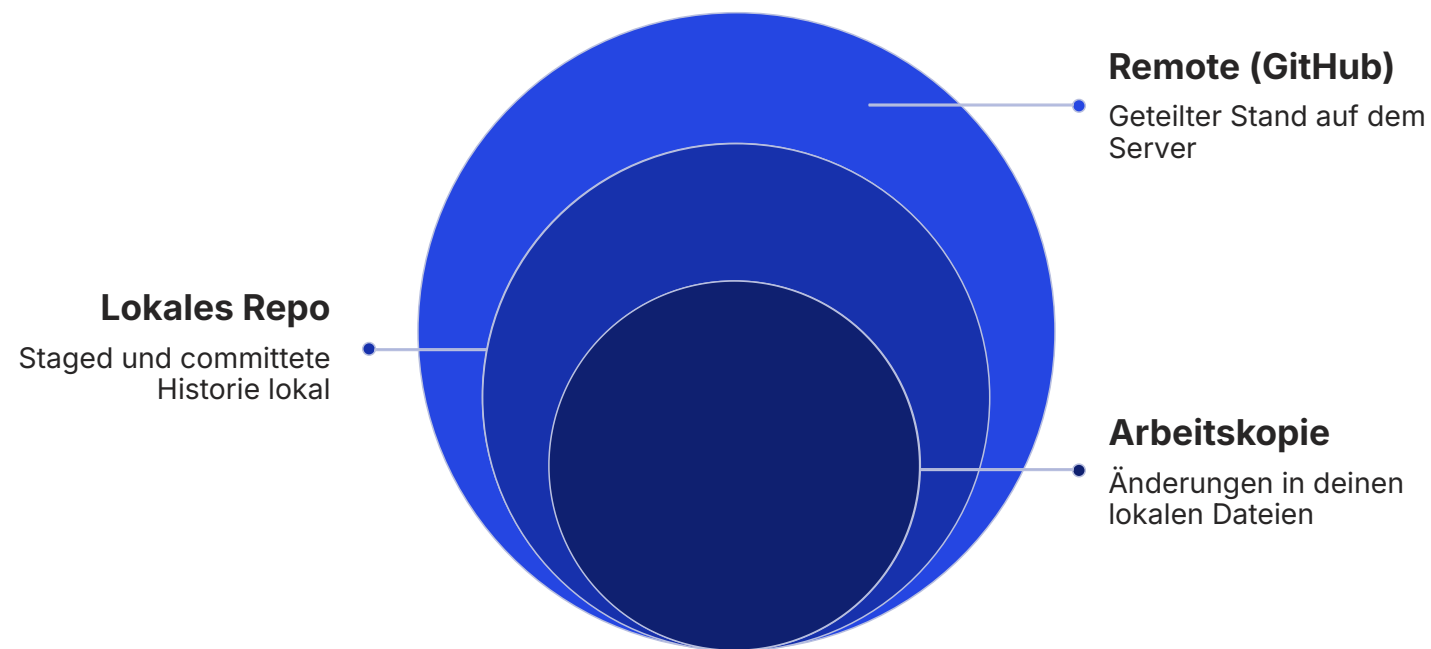
Das System, das Änderungen sauber verfolgt und lokale sowie Remote-Stände koordiniert.

GitHub = Plattform

Die Plattform, auf der du Git-Repositories online verwaltest, teilst und mit anderen zusammenarbeitest.

Drei Stände – sauber trennen

Viele Git-Probleme entstehen nicht, weil Git „kompliziert“ wäre, sondern weil man diese drei Stände nicht klar trennt:



Wer diese drei Ebenen klar im Kopf hat, versteht die meisten Git-Fehlermeldungen sofort besser.

Ein GitHub-Konto anlegen

Damit du GitHub verwenden kannst, brauchst du ein persönliches Konto. Die Registrierung erfolgt direkt über **github.com**.

01

github.com aufrufen & Sign up klicken

02

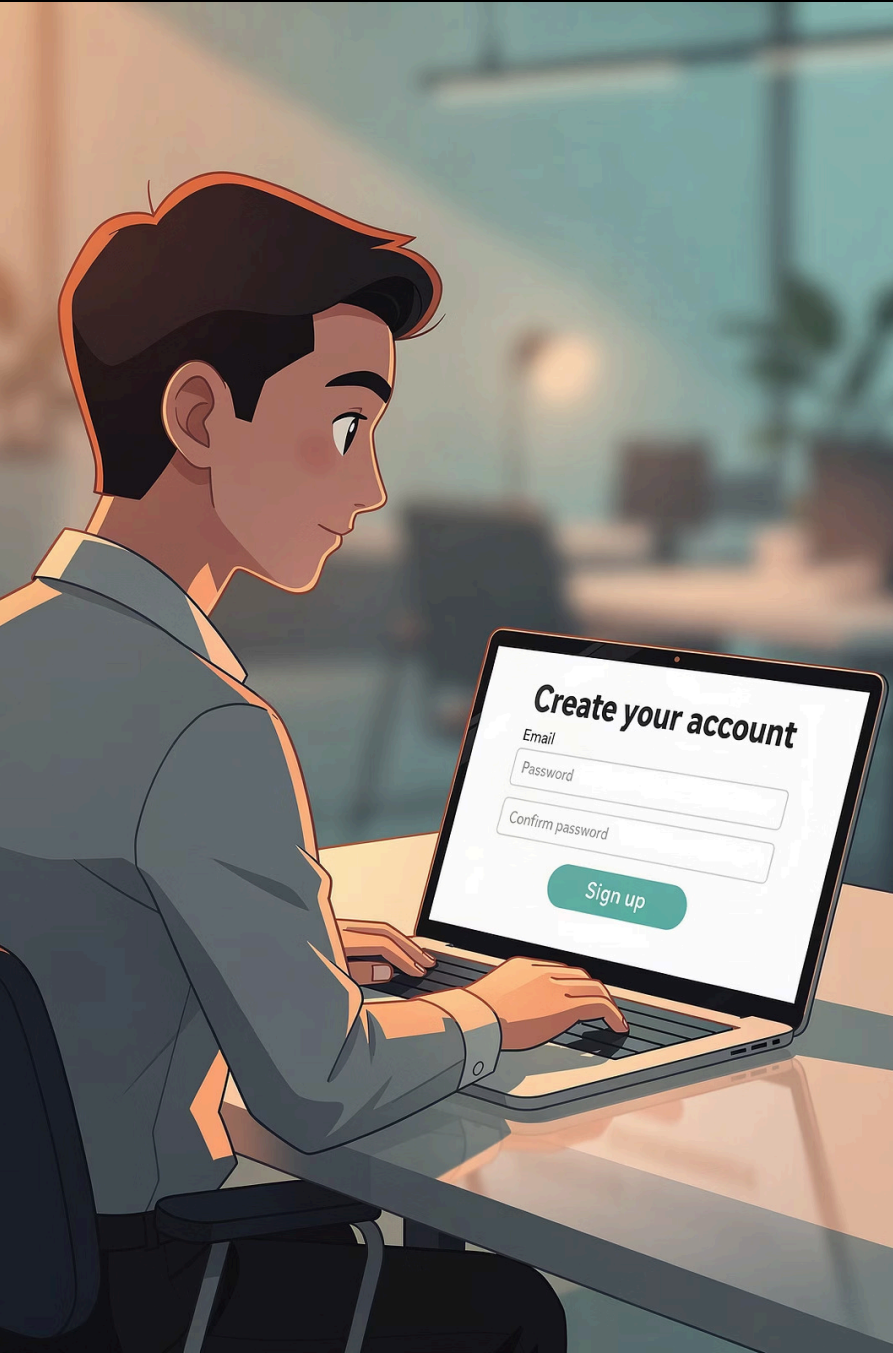
Benutzername, E-Mail und Passwort wählen

03

E-Mail-Adresse bestätigen

04

**Repositories erstellen, Projekte veröffentlichen,
zusammenarbeiten**



Was du mit deinem GitHub-Konto tun kannst



Repositories verwalten

Eigene Repositories anlegen, als **public** oder **private** verwalten und bestehende Projekte online speichern.



Zusammenarbeiten

Mit anderen entwickeln, Pull Requests erstellen und Issues verwalten.



Dokumentieren

Projektdokumentation über README-Dateien direkt im Repository anzeigen.

❏ **Wichtig:** Für Git-Operationen über HTTPS reicht dein GitHub-Passwort nicht aus. GitHub verlangt sichere Authentifizierung via **Personal Access Token**, Browser-Anmeldung oder **SSH-Schlüssel**.

HTTPS oder SSH?

Für die Verbindung zu GitHub gibt es zwei übliche Wege. Beide sind offiziell von GitHub dokumentiert und unterstützt.

HTTPS

Einfach für den Einstieg. Authentifizierung über Token oder Anmeldedialog. Empfohlen für Anfänger.

SSH

Sehr angenehm auf Dauer. Einmal SSH-Key einrichten – danach oft bequemer. Empfohlen für regelmäßige Nutzung.

Wenn du gerade erst anfängst, ist **HTTPS** oft leichter zu verstehen. Wenn du regelmäßig mit GitHub arbeitest, ist **SSH** langfristig oft angenehmer.



VERBINDUNG

Lokales Repository mit GitHub verbinden

Dein lokales Repository kennt GitHub erst dann, wenn du ihm sagst: „Dieses Online-Repository dort ist mein Remote.“ Dafür wird meist der Name **origin** verwendet – ein verbreiteter Standardname, kein magisches Wort.

Typischer Verbindungsablauf

01

Lokal initialisieren

`git init` → `git add .` → `git commit -m "Erster Commit"`

02

Leeres Repository auf GitHub erstellen

Auf github.com ein neues, leeres Repository anlegen (z. B. `mein-projekt`).

03

Remote verbinden

`git remote add origin https://github.com/DEIN-NAME/mein-projekt.git`

04

Ersten Push durchführen

`git branch -M main` → `git push -u origin main`

Mit `git remote -v` kannst du jederzeit prüfen, ob die Verbindung korrekt gesetzt wurde.

Wichtiger Anfängerhinweis: Leeres Repository

✓ Empfohlene Variante A

Erst lokal anfangen und auf GitHub ein **leeres** Repository anlegen – ohne README, .gitignore oder Lizenz.

✓ Empfohlene Variante B

Erst auf GitHub anlegen (mit README etc.) und danach das Repository **klonen** – dann stimmt die Historie von Anfang an.

- Wenn das Remote-Repository bereits einen ersten Commit hat und dein lokales Repository ebenfalls eigene Commits besitzt, weichen die Historien voneinander ab. Der erste Push schlägt dann fehl – das ist eine **Schutzfunktion von Git**, kein Fehler von GitHub.



KLONEN

Ein bestehendes Repository klonen

Klonen bedeutet: Du holst ein bestehendes Repository von GitHub komplett auf deinen Rechner – inklusive der vollständigen Git-Historie. Nicht zu verwechseln mit einem ZIP-Download, der keine Git-Verbindung enthält.

Klonen: Wann und wie

Wann klonen?

- Das Repository existiert bereits auf GitHub
- Du willst an einem fremden oder bestehenden Projekt arbeiten
- Du brauchst die komplette Git-Historie lokal
- Du willst nicht manuell ein leeres Verzeichnis anlegen

Was nach dem Klonen automatisch da ist

- Lokaler Projektordner
- Vollständige Commit-Historie
- Standard-Remote `origin` (automatisch eingerichtet)
- Ausgecheckter Standard-Branch

Befehl: `git clone https://github.com/DEIN-NAME/mein-projekt.git` – oder per SSH: `git clone git@github.com:DEIN-NAME/mein-projekt.git`

SYNCHRONISATION

Synchronisation mit GitHub

Synchronisation bedeutet, dass dein lokaler Stand und der Stand auf GitHub aufeinander abgestimmt werden. In Git synchronisierst du vor allem **Commits** und **Branch-Stände** – nicht einfach lose Dateien.

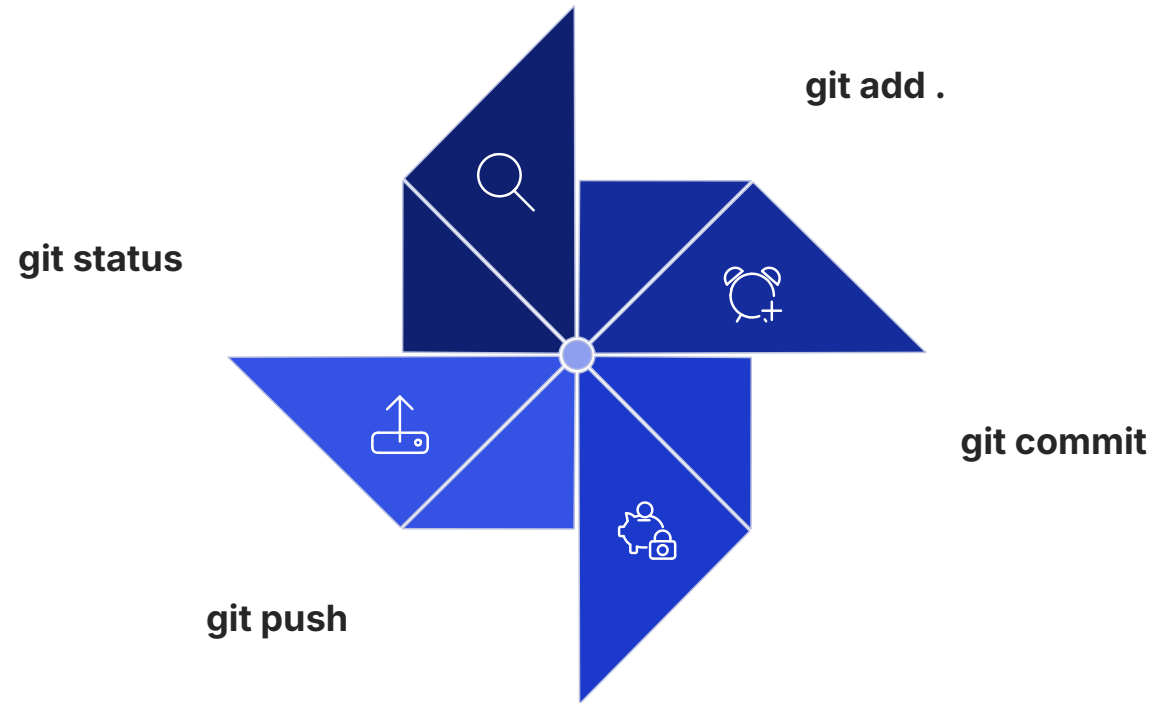
↑ Lokal → GitHub

Du sendest deine Änderungen hoch: `git push`

↓ GitHub → Lokal

Du holst Änderungen herunter: `git fetch` oder `git pull`

Änderungen hochladen: Der Standardablauf



❏ **Häufiger Denkfehler:** `git add` und `git commit` laden noch nichts zu GitHub hoch. Erst `git push` überträgt Commits zum Remote. Diese Trennung ist zentral.

Wenn git push nicht funktioniert

Häufige Fehlermeldungen und ihre Bedeutung:

rejected / non-fast-forward

Auf GitHub gibt es bereits Änderungen, die du lokal noch nicht hast. Zuerst den Remote-Stand holen und integrieren.

authentication failed

Du bist nicht korrekt authentifiziert.
Kein Passwort mehr möglich – Personal Access Token oder SSH erforderlich.

Kein Upstream gesetzt

Beim ersten Push eines neuen Branches: `git push -u origin feature-login` setzt die Verknüpfung.

fetch vs. pull – ein wichtiger Unterschied

git fetch

Holt neue Commits vom Remote, integriert sie aber **nicht automatisch** in deinen Branch. Die saubere, vorsichtige Variante – du kannst erst prüfen, was sich geändert hat.

```
git fetch origin
```

git pull

Kombination aus fetch und merge: Holt Änderungen und **baut sie direkt ein**. Praktisch, aber manchmal zu automatisch für Lernzwecke.

```
git pull origin main
```

Empfohlener sicherer Ablauf: `git fetch origin` → `git status` → `git pull origin main`

ZUSAMMENARBEIT

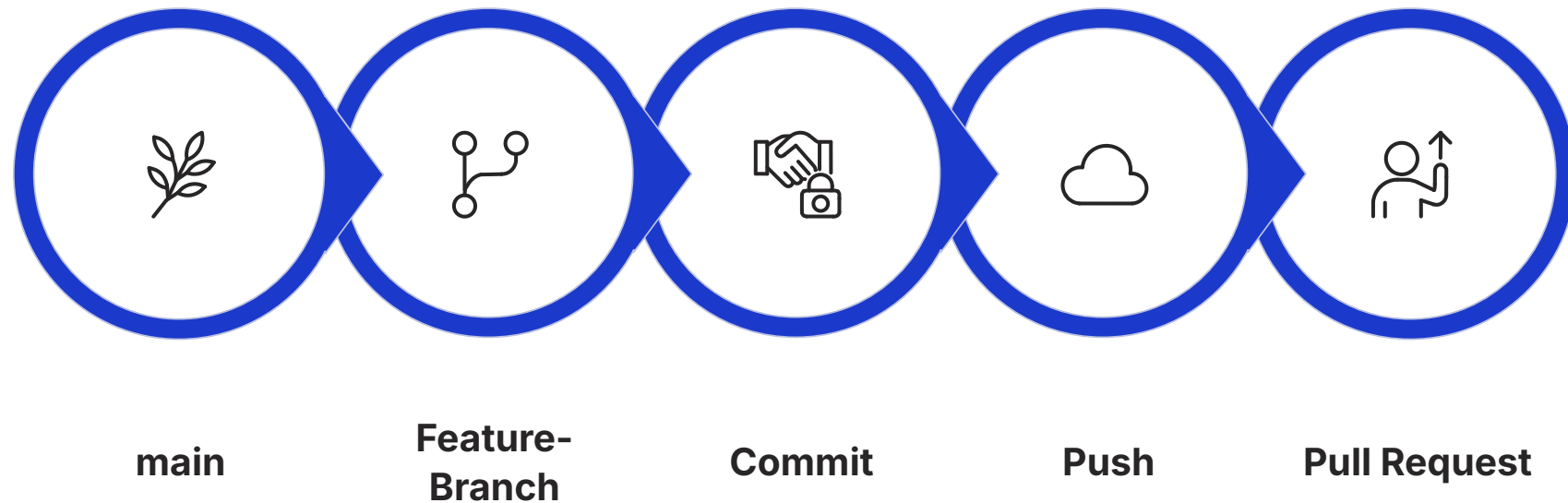
Zusammenarbeit mit GitHub

GitHub ist nicht nur ein Speicherort für Code – die Plattform ist vor allem für Zusammenarbeit gebaut. Wer Branches, Pull Requests und Merge-Vorgänge versteht, versteht den eigentlichen Mehrwert von GitHub.



Pull Requests: Grundprinzip

Ein **Pull Request** ist der übliche GitHub-Weg, um Änderungen aus einem Branch in einen anderen vorzuschlagen – zur Prüfung, Diskussion und anschließendem Merge.



Ein Pull Request trennt **Code schreiben** von **Code freigeben** – das macht Änderungen überprüfbar und den Hauptbranch stabiler.

Was du in einem Pull Request siehst



Dateiänderungen

Welche Dateien und Zeilen hinzugefügt oder entfernt wurden – übersichtlich hervorgehoben.



Reviews & Kommentare

Wer was kommentiert hat, ob Checks erfolgreich waren und ob Konflikte vorliegen.



Merge-Status

Ob der Branch konfliktfrei gemergt werden kann und welcher Branch das Ziel ist (**Base** vs. **Compare** Branch).

- ❑ Selbst wenn du alleine arbeitest, sind Pull Requests nützlich – sie zwingen dich, deine Änderung als abgeschlossene, überprüfbare Einheit zu betrachten.

Merge-Konflikte erkennen und lösen

Ein **Merge-Konflikt** entsteht, wenn dieselben Zeilen in konkurrierenden Branches unterschiedlich bearbeitet wurden. Git markiert den Konflikt in der Datei:

```
<<<<<< HEAD
Hallo Welt
=====
Hallo GitHub
>>>>>> feature-begrueßung
```

Was die Marker bedeuten

```
<<<<<< HEAD = Version deines aktuellen Branches
===== = Trennlinie
>>>>>> = Version aus dem anderen Branch
```

Lösung in 3 Schritten

1. Datei manuell bearbeiten – gewünschten Inhalt behalten
2. Alle Konfliktmarker entfernen und speichern
3. `git add` → `git commit`

Konflikte vermeiden: Gute Lernregel

Je länger ein Branch vom Hauptbranch getrennt bleibt, desto größer wird das Konfliktrisiko.

→ **Kleinere Änderungen machen**

Überschaubare, fokussierte Commits reduzieren Überschneidungen mit anderen Branches.

→ **Häufiger synchronisieren**

Regelmäßig `git pull` oder `git fetch` ausführen, um den lokalen Stand aktuell zu halten.

→ **Pull Requests nicht ewig offen lassen**

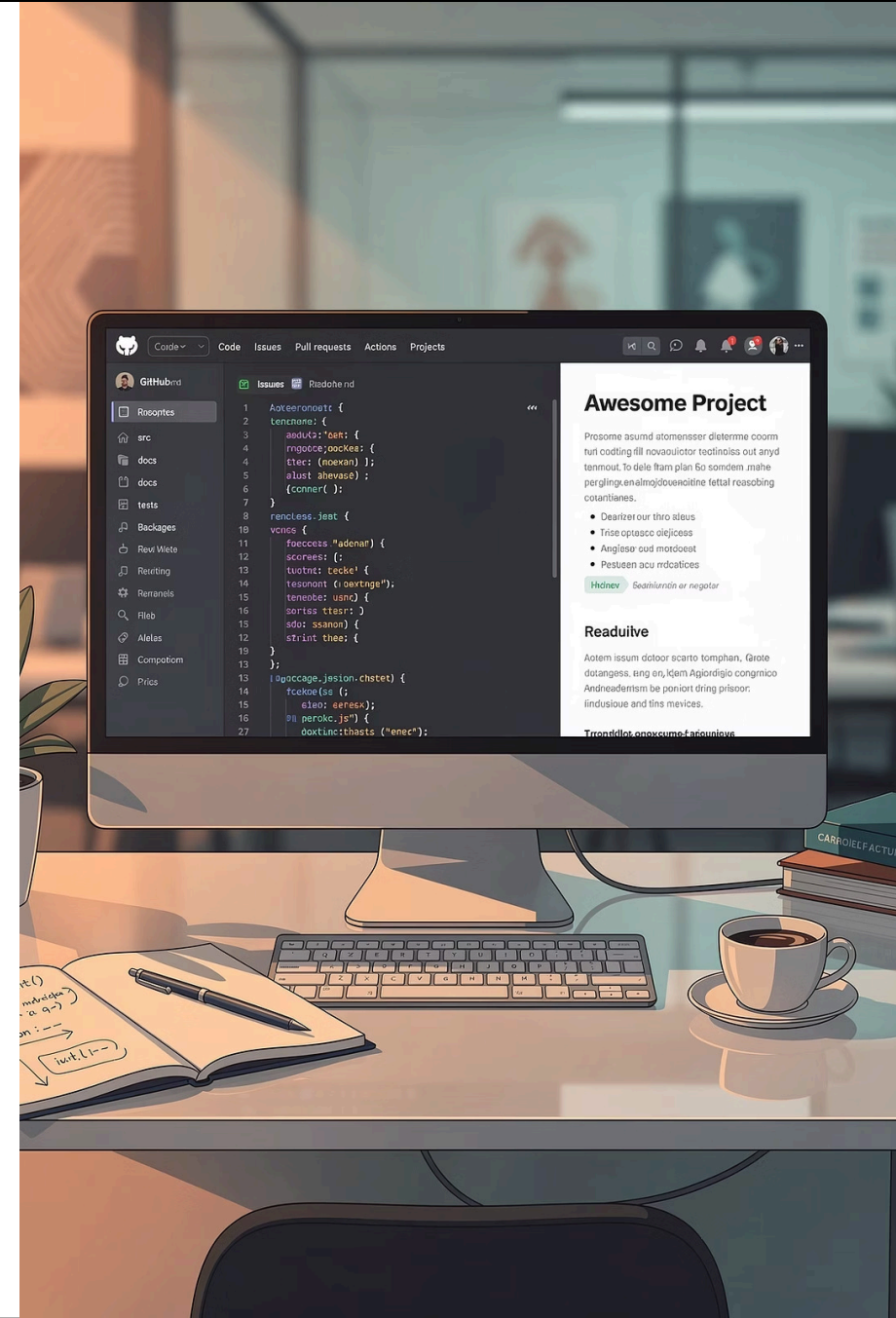
Lange offene Branches divergieren stärker – frühzeitiger Merge reduziert Konflikte erheblich.

DOKUMENTATION

Dokumentation auf GitHub

Ein gutes Repository besteht nicht nur aus funktionierendem Code.

Dokumentation ist nicht „Extra-Arbeit“, sondern Teil guter Softwareentwicklung – sie hilft dir selbst genauso wie anderen.



Navigation in der GitHub-Weboberfläche

Wenn du ein Repository auf GitHub öffnest, findest du mehrere zentrale Bereiche:

Code	Dateien, Ordner, Branch-Auswahl, Klonen, Commits
Issues	Aufgaben, Fehlerberichte, Diskussionen
Pull requests	Änderungsvorschläge und Reviews
Actions	Automatisierungen und CI/CD-Workflows
Projects	Projektorganisation
Wiki	Zusätzliche Dokumentation
Insights	Statistiken und Aktivitätsdaten

- ☐ Wer GitHub nur als „Ort zum Hochladen“ sieht, nutzt vielleicht 20 % der Plattform. Code, Issues und Pull Requests sind besonders zentral.

Issues vs. Pull Requests

Issue

Ein Problem, eine Aufgabe oder eine Idee. Kein Code – sondern eine Diskussion oder ein Auftrag. Wird unter dem Reiter **Issues** verwaltet.

Pull Request

Eine konkrete Codeänderung als Vorschlag. Enthält Commits, Dateivergleiche und Review-Kommentare. Wird unter **Pull requests** verwaltet.

Ein häufiger Anfängerfehler ist, Issues mit Pull Requests zu verwechseln. Beide sind wichtige Werkzeuge – aber für unterschiedliche Zwecke.

README.md: Die Einstiegsdokumentation

Die Datei **README.md** ist die Standard-Einstiegsdokumentation eines Repositories. GitHub zeigt sie prominent auf der Startseite an. **README** bedeutet sinngemäß „Lies mich zuerst“, **.md** steht für Markdown.

Was ist das Projekt?

Name, Beschreibung und Zweck des Projekts klar benennen.

Wie startet man es?

Installations- und Starthinweise, damit andere das Projekt lokal zum Laufen bringen.

Wie nutzt man es?

Wichtige Nutzungsschritte, verwendete Technologien und aktueller Projektstatus.

Wichtige Markdown-Bausteine

Markdown ist bewusst einfach gehalten. Ein paar Bausteine reichen schon weit:

# Überschrift	Große Überschrift (H1)
## Unterüberschrift	Kleinere Überschrift (H2)
- Punkt	Aufzählung
`code`	Inline-Code
```bash ... ```	Codeblock
[Text](URL)	Link

- ❑ **Lerneffekt:** Sobald du etwas verständlich dokumentieren kannst, hast du es meist deutlich besser verstanden. Gerade bei Lernprojekten lohnt sich eine gute README besonders.

# Gute Reihenfolge beim Dokumentieren

So wird dein Repository auf GitHub nicht nur technisch korrekt, sondern auch für andere wirklich lesbar und benutzbar.



---

**Projektname & Kurzbeschreibung**



---

**Zweck des Projekts**



---

**Start- und Installationshinweise**



---

**Wichtige Nutzungsschritte**



---

**Projektstruktur oder Besonderheiten**

# Zusammenfassung: Die wichtigsten Konzepte

1

## Lokal vs. Remote

Drei Stände sauber trennen: Dateien, lokales Repo, GitHub.

2

## Verbinden & Klonen

`git remote add origin` oder `git clone` – je nach Ausgangssituation.

3

## Push & Pull

`add` → `commit` → `push`. `Fetch` vor `Pull` für mehr Kontrolle.

4

## Pull Requests & Konflikte

Code schreiben von Code freigeben trennen. Konflikte früh erkennen und lösen.

5

## README.md

Dokumentation ist Teil guter Entwicklung – klar, nützlich, aktuell.